Introduction, Motivation and SWE Practices

Good morning TA, today we will be explaining HabitatHero, a personalized HDB Recommender.

Agenda.

We provide configurable algorithm based on users choice.

To understand our Software Engineering practices, we have to start with our System Architecture. We implemented a strict 3-Tier Layered Architecture. As you can see on the screen, our React Client Tier click is completely decoupled from our Data Tier click. All communication flows strictly through our Spring Boot Middle Tier click.

Within that Middle Tier, we strictly applied the **Boundary-Control-Entity (BCE)** pattern to separate our concerns.

- Click Our **Boundary** layer, utilizing Spring @RestController, acts purely as an API gateway.
- Click Our **Control** layer contains the core orchestration, such as the RecommendationEngine, which handles the heavy lifting.
- Click Finally, our **Entities** represent our data models, keeping state decoupled from the business logic.

This strict separation ensures that our Control layer doesn't have to worry about raw database queries. Instead, it relies on our Data Tier—powered by PostgreSQL and PostGIS—using the Repository Pattern, which leads me to the specific design patterns we used to glue this all together…

[How Layers Interact: IOC, DI, Repository]

Now I will show how our backend layers communicate through Spring Inversion of Control containers and the Repository pattern.

In RecommendationEngine, core external dependencies are injected through the constructor click, so the service does not create its own collaborators and stays focused on business logic. Within that service flow, data is retrieved through repository method calls such as click findAveragePsfByTowns, rather than embedding SQL in the service layer.

On the repository slide, ResaleTransactionRepository extends JpaRepository and defines query contracts with @Query methods click. So persistence logic is centralized in the repository layer, while services orchestrate domain logic.

This separation gives lower coupling, clearer responsibilities, easier testing, and better maintainability.

[Strategy Pattern]

Here we implemented the Strategy pattern for ranking recommendations.

In the top snippet, RecommendationEngine receives a click RankingStrategy through constructor injection click, so the engine depends on the interface, not a hardcoded sorting rule. Then, after finalizing candidate results, it delegates the final ordering through click rankingStrategy.rank(...).

In the bottom snippet, click GlobalMatchRankingStrategy is one concrete strategy implementation. It defines the ranking policy: higher global match first, then lower price, then higher remaining lease.

So if we want a different ranking behavior later, we can add another strategy class without changing RecommendationEngine, which follows Open-Closed and keeps the engine maintainable.

[Facade Pattern]

Here we apply the Façade pattern using MainSpatialMgr click. This class gathers multiple geospatial managers, like click HDBBuildingMgr, TransportLineMgr, and LandUseMgr, into one place. Instead of letting controllers call each subsystem directly, clients call MainSpatialMgr methods like click getNoiseLevel and click getSunFacing. So MainSpatialMgr hides internal complexity and gives a single, consistent entry point. That reduces coupling, improves maintainability, and localizes subsystem changes to the facade layer.

Traceability

We showcase traceability from the use case diagram to the calSunFacing() method, which performs calculations for sun-facing analysis, containing the core algorithm.

The process begins with the Generate Recommendation use case, which includes calculating match scores.

In its use case description, In step 2, the user loads available ranked HDB block results from the recommendations.

In the class diagram, the backend logic starts at RecommendationController, which has RecommendationEngine class and recommend method.

The recommend() method in the controller is called via a backend POST API Call and invokes RecommendationEngine.generateRecommendation().

which can be seen in the class diagram for recommendation engine.

This method filters results, calculates match scores, and computes commute scores.

For match scoring, the SunFacingResult and subsequent solar score is retrieved via getSunFacingResult(),

which calls getSunFacing() in HDBBuildingMgr.

This can be seen in its class diagram

If results aren't found, it computes the score at runtime using calSunFacing().

In GeoSpatialAnalysis/src, Singleton design principles are evident. For example, HDBBuildingDbMgr, it maintains a single instance accessed via getInstance(), ensuring consistency and simplified global access.

The prevalence of small classes reflects the Single Responsibility Principle (SRP). For instance, HDBBuildingGeoJsonDownloader only downloads HDB building datasets from data.gov via DataGovAPIHandler. Future changes to download method/source require modifying only this class. Hence, SRP improves maintainability, readability, reduces coupling, and supports scalability by isolating changes and allowing new classes to be added without affecting existing ones.

Testing is done on the singleton implementation in GeoSpatialAnalysis/src, where for every class with the getInstance() method, it is called twice and the object reference for each retrieval is compared and ensured that it is the same object, same instance. Testing results showed successful implementation of singleton principles.

Persona 1

Let me start by introducing the first persona for our demo. This persona represents a retired couple looking to downsize to a 2-room flat after their children have moved out, preferably in mature estates such as Ang Mo Kio, Bishan, and Toa Payoh. Their key needs include being within walking distance to hawker centres and supermarkets, avoiding strong west-facing afternoon sun, and staying in a quieter environment with minimal MRT or LRT noise. Their budget is tight, as they need to support long-term retirement living. Based on these requirements, I'll now move on to the demo to show how our system generates suitable housing recommendations for this couple.

First, I'll create a new account by entering my name, email and password. Once the account is created, I'll log in using the same credentials to access the system. To generate suitable housing recommendations, I will click on the start lifestyle quiz button.

Starting with the structural constraints, for the budget, I'll set it to between $200k to $300k since they are more price-sensitive and saving for retirement. For the preferred region, I'll select North-East and Central, and under towns, I'll choose Ang Mo Kio and Bishan for North-East, and Toa Payoh for Central. The flat type will be a 2-room flat, and for the minimum lease, I'll set it to 50 years.

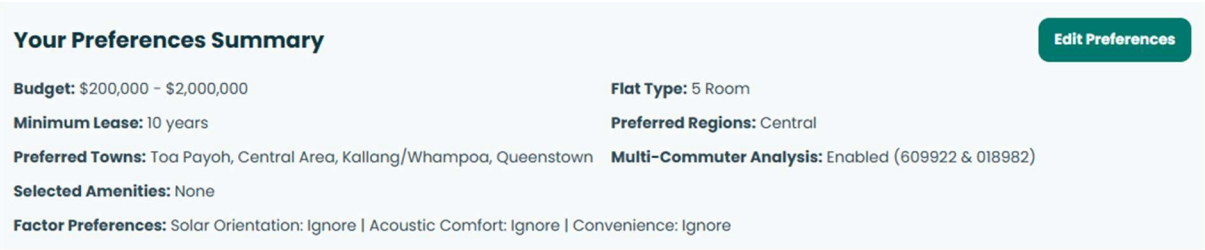
Moving on to livability factors, I'll set all three — solar orientation, acoustic comfort, and convenience, as strict requirements. For selected amenities, I'll select hawker centres and supermarkets, based on the needs of the person. For the multi-commuter analysis, I'll leave it disabled since it’s not relevant in this case.

After selecting all the preferences, I will go ahead and click on “find my HDB match” to generate the recommendations. Based on the given preferences, the system returns 11 matching HDB blocks tailored to this profile. I'll select the first one since it has a 100% match.

On the map, the blue pin shows the location of the HDB block. Below it, there’s a sun icon indicating the building’s orientation. If I click on it, I can see more details — for this block, the heat exposure is relatively low at 32.1%, which means it is well-shielded from strong afternoon sun. The green circular area represents the noise buffer. Here, the nearest MRT is about 1315 metres away, with a noise level of 36.6 dBA, which is within comfortable residential limits.

Looking at nearby amenities, there are icons showing a hawker centre and a supermarket close to the block. The nearest hawker centre is NTUC Foodfare, about 588 metres away, and the nearest supermarket is Giant, about 330 metres away. For more detailed accessibility, we can refer to the pedestrian access section below. It shows that Giant is about a 4-minute walk, while NTUC Foodfare is around a 7-minute walk from the block.

Persona 2



Next, let's look at Persona 2. They are a dual-income couple with a healthy budget of up to $2 million. However, one works in Jurong, and the other works in the CBD. Their main priority is finding a 5-Room flat in a Central estate that offers a perfectly fair and efficient daily commute for both of them

Because commute fairness is their absolute top priority right now, they set the environmental soft constraints—like Solar Orientation and Acoustic Comfort—to 'Ignore'.

When we hit generate, our backend MultiCommuterService sends concurrent requests to the OneMap Routing API to get highly accurate, real-world travel times. However, we engineered this system for enterprise-grade robustness. We enforce a strict 800-millisecond timeout limit. If the external API is down or lagging, our system catches the error and instantly falls back to a Haversine straight-line distance estimate. This guarantees the UI never hangs and the user always receives a 'best-guess' result.

For the MultiCommuterService weight, a higher weight emphasises on making the 2 distances equal while a lower weight emphasises on making the combined distance smaller. So we choose 0.7 because we want to slightly prioritise making the combined distances similar.

Persona 3

Let’s take a look at Persona 3, which is a young family with a 5-year-old child looking for a 5-Room flat with enough space for a growing child. Their top priority is to stay in the East while staying near a primary school for their child.

In terms of accessibility, they must be close to the child’s school and within convenient reach of supermarkets, schools, and playgrounds for daily family needs. As for their comfort, they must avoid strong west afternoon sun. Acoustic comfort is important but slightly flexible. The flat must be affordable for a growing family while supporting the costs of a larger flat and long-term child-raising expenses.

Let’s move on to the live demo. Budget will be 400-700k, for a 5-Room flat of minimally 50 years left in its lease. Preferred region in the East, specifically Tampines or Pasir Ris. As for factor preferences, solar orientation is strict, acoustic comfort is weighted at 0.5, convenience is strict, where the flat needs to be near supermarkets, schools, and playgrounds. Multi-commuter analysis is disabled. Then, we generate preferences.

System Robustness and Compliance

Moving on I will be talking about our system robustness and compliance which ensures HabitatHero is running smoothly.

1) Data persistence

To ensure a seamless user experience, we implemented stateless authentication. Notice that even after a hard refresh or closing the browser, I am still logged in. This is because our React frontend persists a JSON Web Token (JWT) in the browser's local storage.

2) Password Change & Security

We also implemented a password change feature. For security, we never store plain-text passwords. We utilize BCrypt hashing on the backend. This ensures that even in the event of a database compromise, actual user passwords remain unrecoverable."

3) Guest account

"We also accounted for guest users. If an unauthenticated user takes the quiz, they can switch between tabs without losing their data. However, unlike registered users, their results aren't saved to the database and will clear upon a browser refresh.

4) Input Validation

We implemented robust input validation to ensure data integrity. The system instantly flags invalid entries and provides descriptive prompts to guide the user through correcting the error.

5) Error Handling

This leads into our Error Handling. We designed specifically for 'Edge Cases.' If a user's search criteria are too strict, we avoid showing a blank screen or crashing. Instead, we provide a Graceful Error State that encourages the user to adjust their parameters, maintaining high usability even when no matches are found.

6) The Legal & Ethics Page

As per our non-functional requirements in our SRS, we have also included a comprehensive Legal Disclaimer and address PDPA compliance, ensuring users know exactly how their saved profiles are handled.

7) Resource

Moving on we also have a Housing Resources hub, that is a curated knowledge base that integrates official SLA, HDB, and CPF guidelines.

8) Responsive Design

Finally, HabitatHero is fully responsive. Using CSS Flexbox, we ensured the app works across all devices. You'll notice the map automatically scales its viewport to remain fully functional even on smaller mobile screens."